LumiNovels: Deep Learning-Powered Light Novel Recommendations

MD Tarek Islam

Department of Computer Science, University of Warwick

Progress Report

1 Introduction

LumiNovels endeavours to restructure recommendations by addressing the challenge of limited accessibility in Western markets. Traditional recommender systems face hurdles like accommodating diverse preferences, sparse data and bounded content analysis. Leveraging advanced technology and deep learning, LumiNovels aims to redefine this landscape.

Through a fusion of React, TypeScript and Java, LumiNovels incorporates a sophisticated deep-learning algorithm fuelled by MyAnimeList and AniList APIs. This project notably offers users flexibility – they can seek recommendations based on chosen genres and previous reads or submit synopses. The innovation lies in employing a hybrid approach of deep neural networks and content-based filtering with natural language processing, breaking free from rigid input requirements.

LumiNovels' objectives encompass overcoming challenges like the Cold Start problem, adapting to diverse preferences, and enriching content analysis. This project promises bespoke and varied recommendations, elevating the light novel reading experience.

1.1 Primary Objectives

- **Utilize MyAnimeList and AniList APIs:** construct a dataset in the form of a CSV file comprising of 10000+ light novels - sourced from the two APIs for robust algorithmic analysis and individual recommendations.
- **Develop Advanced Deep Learning Algorithm:** craft and integrate a sophisticated deep learning algorithm that harnesses neural networks and natural language processing techniques. This algorithm aims to deliver precise light novel recommendations based on multifaced criteria, including genre selection, user preferences, user-provided synopses and previously read light novels.
- **Establish a Robust Java Backend:** create a resilient Java backend through Maven, Spring Boot and the Java Kernel for Jupyter Notebook to house the powerful deep learning model. This backend will efficiently manage the recommendation system, process user queries and execute content analysis.
- **Develop a User-Friendly React App with TypeScript:** produce an engaging and intuitive React App using TypeScript. This frontend will serve as the interface for LumiNovels, facilitating seamless user interactions and ensuring a responsive design for enhanced user experience.
- **Implement Content-Based Filtering Techniques:** fine-tune recommendations with content-based filtering. Enable users to receive precise suggestions aligned with their preferences by considering synopses, user-provided information and genres.

1.2 Secondary Objectives

- **Refine User Interface:** create an intuitive, visually appealing web app that prefers ease of use and incorporates responsive design principles by continuously refining the user interface.

- **Optimize User Experience:** deliver swift response times, seamless navigations and smooth interactions, ensuring optimal performance and enhancing user satisfaction.
- **Enhance User Engagement:** employ interactive features like user ratings, reviews and an internal community platform for readers to discuss recommended light novels, fostering increased user engagement.
- **Real-Time Content Updates and Recommendations:** incorporate such functionalities to provide users with the latest content and personalised suggestions.

1.3 Possible Extensions

- **Sentiment Analysis and Advanced Natural Processing:** explore advanced natural language processing techniques to scan novel summaries, user comments and reviews for generating more nuanced and contextually relevant recommendations.
- **Implement Adaptive User Profiling techniques:** develop advanced user profiling mechanisms that continuously evolve and learn from users' changing preferences; this ensures that recommendations remain relevant and tailored to individual users over time.
- **Enable Social Interactions Within the Platform:** facilitate users to share favourite light novels, recommendations and reviews with friends within the LumiNovels platform – foster community engagement through user discussions and interactions.

2 Background Research

2.1 Literature review

The literature review table below displays various methodologies and approaches to recommender systems. The table provides insights into the diverse strategies to tackle challenges such as recommendation accuracy, domain-specific requirements, cold start problems and data sparsity. This overview provides a panoramic view of different models, their objectives, the datasets involved, and the advantages and disadvantages associated with each approach.

Name	Objective	Methodology	Features	Datasets	Advantages	Disadvantages
A hybrid recommender system using artificial neural networks [1]	Predict movie ratings by combining collaborative filtering and content-based data	Artificial neural network classifier	User demographics, item content features, ratings	MovieLens	Combines multiple data sources, handles sparse data	Computationally expensive to train, prone to overfitting
Collaborative Deep Learning for Recommender Systems [2]	Predict item ratings by integrating deep learning on content with collaborative filtering on ratings	Stacked denoising autoencoder + matrix factorization	Item text content, user-item ratings	CiteULike, Netflix	Learns item representations from content, captures user-item relationships	Complex model, difficult to optimize
LFDNN: A Novel Hybrid Recommendation Model Based on DeepFM and LightGBM [3]	Predict clickthrough rates by combining DeepFM and LightGBM	Shallow FM model, deep neural network, gradient boosting trees	User-item interactions	Avazu, Criteo	Learns high-order feature crosses, handles sparse features	Complex Hybrid Architecture Deep Learning Recommendation Model for Personalization and Recommendation Systems
Wide & Deep Learning for Recommender	Combine memorization and	Joint training of wide linear models and deep neural networks	Sparse features, dense embeddings	Google Play app store data	Improves topical relevance and item	More complex architecture

Systems [4]	generalization abilities for recommender systems				diversity, efficient model serving	
Deep Neural Networks for YouTube Recommendations [5]	Personalized video recommendations on YouTube	Deep candidate generation model and deep ranking model	Video IDs, watch history, search queries, etc.	YouTube video data	Handles cold start, improves diversity and freshness	Computationally expensive, requires huge amounts of data
A Novel Approach for Book Recommendation Systems [6]	Generate association rules among books for recommendations	FPIntersect algorithm	Books purchased by users	Synthetic dataset	Reduces the number of database scans compared to Apriori	Specific to book recommendation
Improving a Hybrid Literary Book Recommendation System through Author Ranking [7]	Improve book recommendations by incorporating author preferences	Weighted hybrid of book and author collaborative filtering	User book ratings, authorship	LitRec	Incorporates author preferences, customizable weighting	Domain specific to literary books
Book Recommendation System Using Opinion Mining Technique [8]	Recommend top ranked books using opinion mining	Rating and polarity calculation	Book features, user reviews and ratings	Amazon book data	Incorporates review sentiment analysis	Focused on ranking rather than personalized recs
Book Recommendation System [9]	Book recommendation using association rules	ECLAT algorithm for frequent itemset mining	Books purchased together	Synthetic transaction data	Single database scan, computationally efficient	Rules may not capture personalized interests
Book Recommendation System through content based and collaborative filtering method [10]	Hybrid book recommendation	Content-based, collaborative, keyword-based filtering	Book content, ratings	Custom website data	Incorporates multiple signals - content, ratings, keywords	Requires building a new book website
Deep Learning Recommendation Model for Personalization and Recommendation Systems [11]	Recommend items by combining deep learning on content with collaborative filtering	Deep candidate generation + deep ranking model	User features, item content features	Custom e-commerce data	Captures high-order feature crosses, custom architecture	Requires large amounts of data
Improving Content-based and Hybrid Music Recommendation using Deep Learning [12]	Improve content-based recommendation	Unified deep learning model	Learned from audio	Proprietary music dataset	Handles cold start, does not need CF	Computationally expensive
Deep content-based music recommendation [13]	Address the cold start problem	CNNs to Predict Latent Factors	Learned from audio	Million Song Dataset	Reasonable recommendation, handles cold start	Cannot predict song popularity
Hybrid recommendation system combined content-based filtering [14]	Improve the accuracy of recommendations	Combination of CF, content-based, SOM CF	Text and genres	MovieLens 100K	Improves accuracy, handles cold start	Computationally expensive, slower

Table 1. Literature Review Table: Deep Neural Networks, Deep Learning, Content-Based Filtering, Recommendation Algorithms

The table above presents an exploration of recommender system methodologies. Each literature displayed distinct methodologies with specific objectives, from predicting rating and enhancing accuracy to handling data sparsity and diversifying recommendations. Researchers leveraged diverse datasets – tailored to different domains like books, movies, e-commerce, and music, employing platforms such as Netflix, YouTube, MovieLens, CiteULike, Amazon and proprietary datasets.

Each approach exhibited unique advantages, such as addressing sparse data, improving diversity, incorporating sentiment analysis, or handling domain-specific challenges. However, these models also exhibited limitations, including complex optimization, computational expenses, overfitting risks and the necessity of substantial data. Moreover, studies focused on niche areas such as literary books, association rule generation, opinion mining, and specific algorithms like FPIntersect and ECLAT, indicate a trend towards tailoring recommendation systems to meet the requirements of specific domains.

2.2 Deep Learning in Recommender Systems

- **Implicit Feature Learning:** Deep learning excels at learning implicit features, which are often challenging to define explicitly. These models can uncover nuanced relationships between users and items, enhancing recommendation accuracy by analysing vast amounts of data.
- **Complex Pattern Recognition:** Deep learning models, especially neural networks, recognize intricate patterns within data. In recommender systems, this capability is crucial for understanding user preferences and item features beyond explicit ratings or attributes. It is also the case where traditional recommender systems also suffer from popularity bias, where users with less interest in popular items receive worse recommendation accuracy than other users [15].
- **Personalization and Adaptability:** these models can adapt and customize recommendations based on real-time user interactions. They continually learn from new data, providing dynamic and bespoke suggestions to users.
- **Handling Unstructured Data:** recommender systems often deal with unstructured data like images, audio, video, or text. Deep learning models such as CNNs (Convolutional Neural Networks) and RNNs (Recurrent Neural Networks) are adept at processing such data, enabling better feature extraction and representation.

2.3 Hybridizing Deep Learning with Content-Based Filtering

- **Addressing Data Sparsity:** hybrid approaches mitigate this issue commonly encountered in collaborative filtering-based systems. Content-based filtering provides additional information about items, especially in scenarios where user-item interactions are limited.
- **Enhanced Recommendation Quality:** by combining deep learning with content-based filtering, the system can leverage the strength of both approaches. Content-based filtering emphasizes item features, aiding in recommending items like those a user has already liked, while deep learning captures intricate user-item interactions.
- **Cold Start Problem Mitigation:** the combination assists in handling the cold start problem. Content-based information enables recommendations for new items or users where collaborative filtering might lack sufficient data.
- **Balancing Serendipity and Accuracy:** content-based filtering can introduce serendipity by recommending items based on their features, even if they differ from a user's past preferences. When combined with deep learning, the system has a balance between accuracy and serendipity, enriching the user experience.

In summary, leveraging deep learning in recommender systems offers superior pattern recognition, adaptability and implicit feature learning. Hybridizing this approach with content-based filtering will enhance recommendation quality, balance recommendation quality with serendipity, mitigate cold start problems and address data sparsity issues. This synergy between deep learning and content-based filtering holds promises in developing more robust and bespoke recommender systems.

3 Advanced Methodology

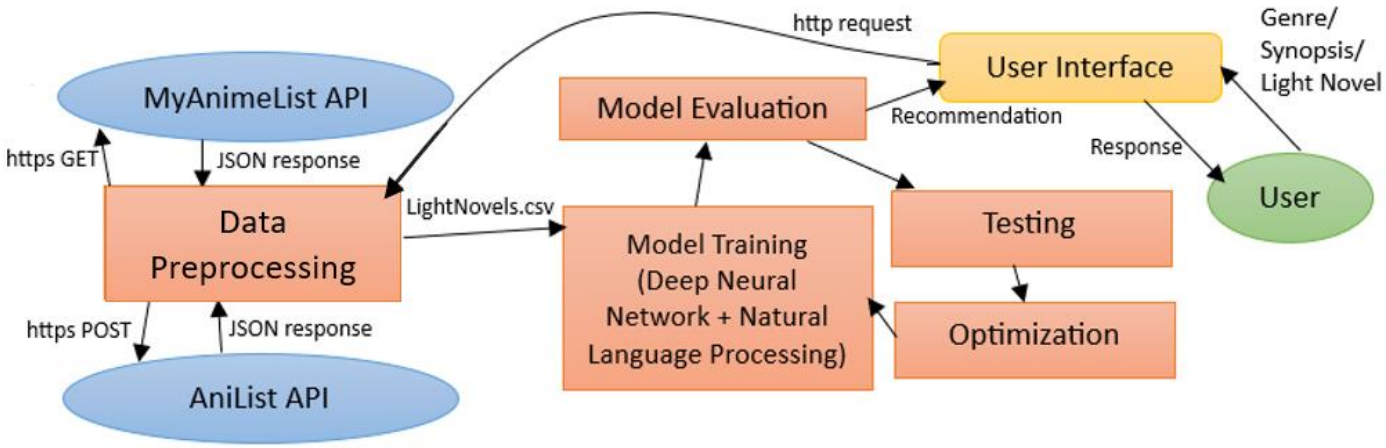


Figure 1. LumiNovels System Diagram

The recommendation system starts with data collection from the MyAnimeList and AniList APIs, through HTTP GET requests for MyAnimeList and HTTP POST requests for AniList, to which the Java code combines the data and adds it to the *LightNovels.csv* file. The preprocessing involves cleaning the raw data from the APIs, addressing missing values, extracting and labelling features which include *id*, *name*, and *synopsis*, and attaining an average score. The user provides a genre, a light novel or a synopsis of their own, which the React frontend accepts and forwards the data to the Java backend via HTTP requests for further processing. The backend oversees user inputs, facilitates backend interactions, and communicates with the stored deep learning model for recommendations.

The trained model for content-based filtering uses the LightNovels.csv data to analyse features like synopsis, genres, and other relevant information, creating representations for light novels with the aid of Natural Language Processing techniques. The Deep Learning model comprehends novel content based on these features and compares user input similarity to dataset novel features to generate recommendations.

The system requires various functionalities to be overcome:

- **Data preprocessing:** cleaning data and preparing it for training.
- **Model selection and training:** choosing the appropriate deep learning architecture and effective training methods.
- **Integration and scalability:** deploying and integrating the trained model into the Java backend while ensuring scalability for efficient user request handling.
- **Evaluation and testing:** determining system performance using relevant metrics.
- **User experience and feedback:** incorporating user feedback, prioritizing frontend user experience to enhance recommendations.

-

4 Progress

4.1 Technical Content

The MyAnimeList (MAL) API documentation became the primary source to follow to collect extensive light novel data. The initial plan was to retrieve JSON nodes to access data on light novels [16], successfully compiling a database of 10,281 distinct entries at the time of this report. However, it consumed a great deal of time, which is not preferable. This resulted in requesting a HTTP GET /manga/ranking method [17], which also avoids the API's potential interpretation of excessive requests as a Denial-of-Service (DoS) attack. By manipulating the manga field to procure top-rated light novels, each request could return up to 500 entries; whereby, implementing pagination using the offset field, all 10000+ entries can be gathered efficiently. The data collected include id, title, images in medium and large sizes, genres, mean and rank scores, popularity metrics, user list and scoring statistics, and synopses. A HashMap structure was used to efficiently organize this dataset, incorporating the id as the key, and storing the associated values as elements in an ArrayList.

Next, as the AniList API was stored as a GraphiQL [18] database, a HTTP POST query was required, utilizing the idMal parameter to attain corresponding details for each light novel in MyAnimeList. The data retrieved from AniList mirrored MyAnimeList's data, with slightly different labels, such as 'idMal' for 'id', 'description' for 'synopsis', 'favorites' for 'num_scoring_users', 'meanScore' for 'mean', and an additional variable, avergaeScore. The scoring system for AniList ranges from 0 to 100, whilst MyAnimeList's scales between 0.0 and 10.0.

The String.split(character) methods were used for data cleaning, removing redundant characters. However, when encountering missing data, if one API contained information which the other did not, that API's data would be stored. MyAnimeList is known for being different – user-written synopses, known for their appeal and relevance, were prioritized. In the case MyAnimeList does not have an entry as a synopsis, the AniList one would be chosen as the backup. A comprehensive genre listing in the final dataset is produces by merging the genres from both APIs. An accurate average score has been computed by employing the Bayesian mean statistical formula [19], amalgamating values including popularity and scoring-related data. At last, the procured dataset was exported into a CSV file via the Jackson Java library [20], labelled with features: id, title, image URLs, genres, score, and synopsis, separated by commas for ease of access.

4.2 Testing

The testing process commenced cautiously with a small sample of 5 light novel entries, such that the data is accurate from both MyAnimeList and AniList APIs. Subsequently, the scope of the process expanded incrementally to 100, and then to 500 entries, with careful testing conducted at each stage to ensure data accuracy, consistency and API functionality. Confidence in the reliability of the MyAnimeList API led to smooth navigation through the extensive repository of more than 10000 light novels. This approach averted inconsistencies or data loss, effectively merging the data retrieved from both MyAnimeList and AniList APIs into a

seamlessly incorporated dataset. The resulting extensive CSV file possesses crucial details for each light novel, poised for further analysis.

5 Project Management

5.1 Reflection

Upon reflection of the progress achieved thus far, despite a slight setback in adhering to the initially suggested Gantt chart schedule, overall satisfaction comes from the accomplishments. Data extraction was successful, despite a one-week delay; feature engineering and the compilation of appropriate light novel information led to a structured CSV file, which is critical for subsequent deep learning model training. This experience has accentuated the efficacy of concurrent coding and research, emphasizing the effectiveness of a balanced approach.

Challenges appeared notably in grappling with limited resources concerning the Jackson Java Library and navigating the documentation of AniList with a lack of intuitive clarity. Overcoming these impediments demanded leveraging support from the programming community; engaging with fellow developers via community forums fostered swift resolution of AniList-related queries within a day. Despite initial struggles with the Jackson Java library, persistence led to discovering a lesser known yet valuable YouTube resource that provided insight into using the library. This process highlighted the supportive and expansive nature of programming communities, stressing the importance of pursuing help across diverse platforms when confronting coding related hurdles. Looking ahead, I acknowledge the significance of actively engaging with multiple communities to streamline troubleshooting processes and ensure smoother progress in future coding endeavours.

5.2 Timetable

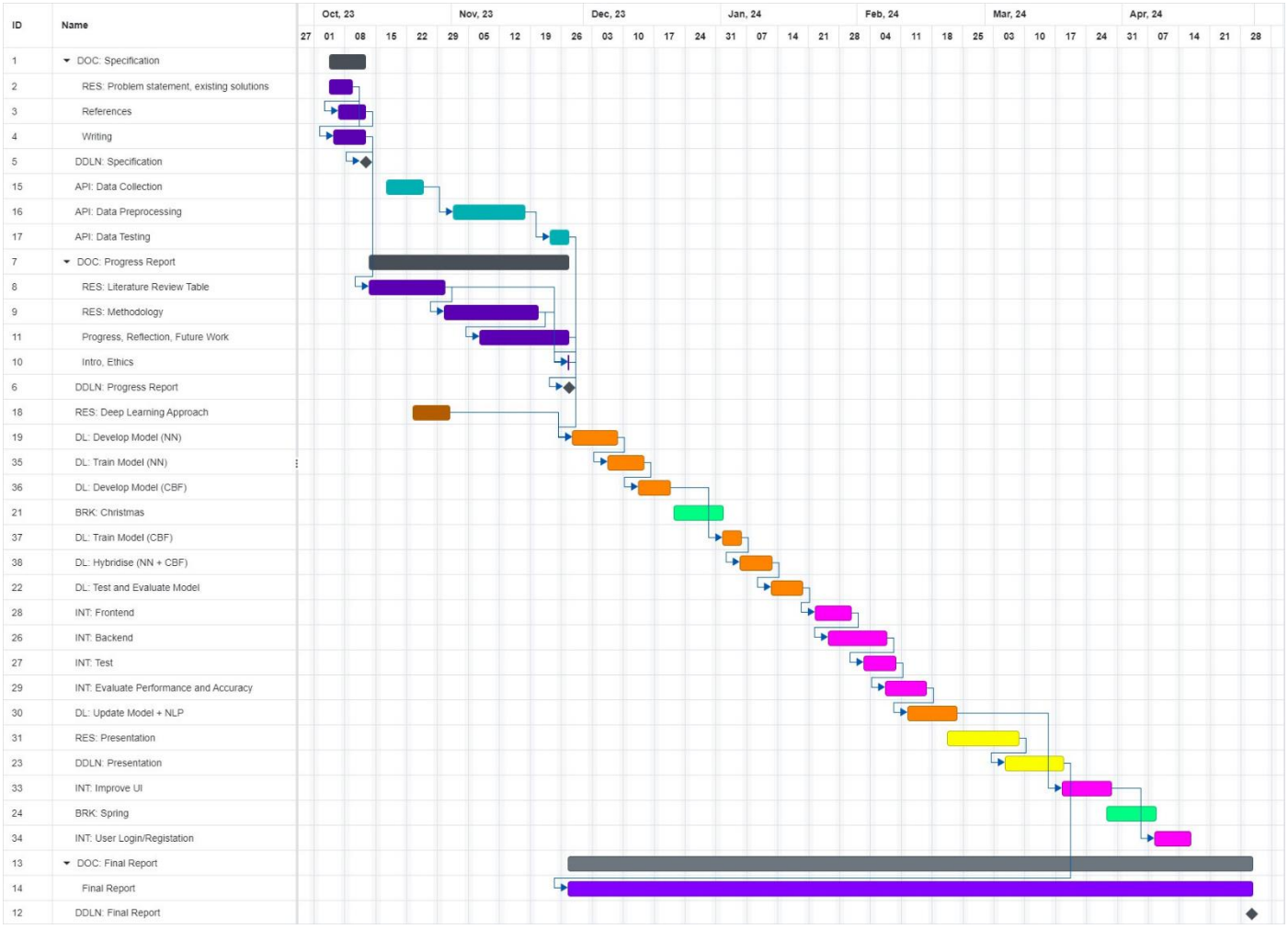


Figure 2. Updated Gantt Chart [21]:
DOC – Document, RES – Research, DDLN – Deadline, API – Application Programming Interface, DL – Deep Learning, NN – Neural Networks, CBF – Content-Based Filtering, BRK – Break, INT – Integration

5.3 Next Term

In the following weeks, the project involves a series of steps centred around the features in the CSV file being used for the development and training of neural networks using Java, leveraging JDK 11, and incorporating Jupyter Notebook with a Python 3 backend alongside a Java kernel. To run Java code seamlessly, the networks will be crafted within the Jupyter Notebook environment. Once the models are produced, the focus will be on training the Content-based filtering algorithm. After rigorous testing and validation of both algorithms, an integration phase is scheduled following the Gannt Chart in *Figure 2*, spanning from week 3 to week 6 of term 2. Within this period, the merging of the algorithms to a singular system will take place, followed by thorough evaluation and testing. Subsequently, attention will be towards finalizing the project's presentation. Weekly updates will be made to the final report, incorporating additional references and details. If time permits, further enhancements will be integrated and considered, including advanced natural language processing, user registration and UI enhancements.

6 Ethics

At present, the project remains exempt from ethical consent requirements, as it does not involve user data, thereby mitigating legal concerns. The APIs are freely accessible under their respective documentation and not for commercial purposes [22, 23]. Moreover, the publicly available data obtained through these APIs do not hold sensitive information. Additionally, the libraries integrated into the project are open source and can be used freely without ethical or legal ramifications.

7 References

1. Paradarami, T. K., Bastian, N. D., & Wightman, J. L. (2017). A hybrid recommender system using artificial neural networks. Expert Systems with Applications, 83, 300-313.

2. Wang, H., Wang, N., & Yeung, D. Y. (2015). Collaborative deep learning for recommender systems. In Proceedings of the 21th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (pp. 1235-1244).

3. Han, H.; Liang, Y.; Bella, G.; Giunchiglia, F.; Li, D. LFDNN: A Novel Hybrid Recommendation Model Based on DeepFM and LightGBM. Entropy 2023, 25, 638.

4. Cheng, H. T., Koc, L., Harmsen, J., Shaked, T., Chandra, T., Aradhye, H., Anderson, G., Corrado, G., Chai, W., Ispir, M., Anil, R., Haque, Z., Hong, L., Jain, V., Liu, X., & Shah, H. (2016). Wide & deep learning for recommender systems. In Proceedings of the 1st Workshop on Deep Learning for Recommender Systems (pp. 7-10).

5. Covington, P., Adams, J., & Sargin, E. (2016). Deep neural networks for YouTube recommendations. In Proceedings of the 10th ACM Conference on Recommender Systems (pp. 191-198).

6. Devika, P., Jisha, R.C., & Sajeev, G.P. (2016). A Novel Approach for Book Recommendation Systems. 2016 IEEE International Conference on Computational Intelligence and Computing Research.

7. Vaz, P.C., de Matos, D.M., Martins, B., & Calado, P. (2012). Improving a Hybrid Literary Book Recommendation System through Author Ranking. arXiv:1203.5324v1.

8. Sohail, S.S., Siddiqui, J., & Ali, R. (2013). Book Recommendation System Using Opinion Mining Technique. 2013 IEEE International Conference on Computational Intelligence and Computing Research (pp. 1609-1614).

9. Mathew, P., Kuriakose, B., & Hegde, V. (2015). Book Recommendation System through Content Based and Collaborative Filtering Method. 2015 International Conference on Soft-Computing and Networks Security.

10. Malhotra, P., Rajpurkar, S., & Bhatt, D. (2015). Book Recommendation System. International Journal for Innovative Research in Science and Technology, 1(11), 314-316.

11. Naumov, M., Mudigere, D., Shi, H. M., Huang, J., Sundaraman, N., Park, J., Wang, X., Gupta, U., Wu, C. J., Azzolini, A. G., Dzhulgakov, D., Mallevich, A., Cherniavskii, I., Lu, Y., Krishnamoorthi, R., Yu, A., Kondratenko, V., Pereira, S., Chen, X., Chen, W., Rao, V., Jia, B., Xiong, L., & Smelyanskiy, M. (2019). Deep Learning Recommendation Model for Personalization and Recommendation Systems. arXiv preprint arXiv:1906.00091.

12. Wang, X., & Wang, Y. (2014). Improving content-based and hybrid music recommendation using deep learning. In Proceedings of the 22nd ACM international conference on Multimedia (pp. 627-636).

13. Oord, A. V. D., Dieleman, S., & Schrauwen, B. (2013). Deep content-based music recommendation. In Advances in neural information processing systems (pp. 2643-2651).

14. Afoudi, Y., Lazaar, M., & Al Achhab, M. (2021). Hybrid recommendation system combined content-based filtering and collaborative prediction using artificial neural network. Simulation Modelling Practice and Theory, 113, 102375.

15. Kowald, D., Mayr, G., Schedl, M., & Lex, E. (n.d.). A Study on Accuracy, Miscalibration, and Popularity Bias in Recommendations. https://arxiv.org/pdf/2303.00400v1.pdf [Accessed 27-11-2023]

16. MyAnimeList API (beta ver.). (n.d.). Myanimelist.net. https://myanimelist.net/apiconfig/references/api/v2#operation/manga_manga_id_get [Accessed 26-11-2023]

17. MyAnimeList API (beta ver.). (n.d.). Myanimelist.net. https://myanimelist.net/apiconfig/references/api/v2#operation/manga_ranking_get [Accessed 26-11-2023]

18. Getting Started - AniList APIv2 Docs. (2023). Gitbook.io. https://anilist.gitbook.io/anilist-apiv2-docs/overview/graphql/getting-started [Accessed 26-11-2023]

19. Do not just rank your results by average rating, use Bayesian Average instead. (n.d.). Arpit Bhayani. https://arpitbhayani.me/blogs/bayesian-average/ [Accessed 25-11-2023]

20. Jackson Tutorial | Baeldung. (2023) Baeldung.com. https://www.baeldung.com/jackson [Accessed 25-11-2023]

21. Free Online Gantt Chart Software. (n. d.) Onlinegantt. www.onlinegantt.com https://www.onlinegantt.com/#/gantt [Accessed 26-11-2023]

22. API License and Developer Agreement. (n.d.). Myanimelist.net. https://myanimelist.net/static/apiagreement.html [Accessed 26-11-2023]

23. API Terms of Use - AniList APIv2 Docs. (2023). Gitbook.io. https://anilist.gitbook.io/anilist-apiv2-docs/overview/overview [Accessed 26-11-2023]

Appendix A

Specification

1 Problem

1.1 Introduction

Light novels are literature of Japanese origin, which provide concise, engaging narratives often accompanied by manga (Japanese comic) style illustrations, catering to diverse tastes and genres. In the Western market, discovering suitable light novels can be challenging - due to their limited availability and relatively small community of enthusiasts. Recommender systems are the solution, as they will efficiently connect readers with literature that aligns with their preferences, enhancing the overall reading experience.

1.2 Current Issues

Recommender systems use algorithms to analyse user interactions, data, and content characteristics to provide personalised suggestions. However, there are many challenges to face:

- **Grey Sheep** are the users whose preferences do not align with any specific group or
  category, leading to difficulty in providing suitable recommendations.
- **Cold Start** is the case of introducing new light novels or users, which may lack data,
  resulting in inaccuracy.
- **Sparsity** occurs from a lack of user ratings, especially with the light novel community being small, giving the collaborative filtering system a challenge. [1]
- **Limited Content Analysis** happens due to a lack of available content data, so content-based recommenders over-recommend similar items, lacking diversity.
- **Evaluation** and **Availability of Online Datasets** prove to be another problem as determining suitable metrics is complex, along with the limited datasets of the light novels being present online. [2]

1.3 AniBrain

AniBrain [3] is a light novel recommender web app that combines collaborative user and content-based filtering to provide suggestions. Users can request random or similar light novels, allowing the adjustments of filters including “story similarity” and “genre similarity”. However, the user must provide an initial light novel, limiting its utility to newcomers to light novels. The goal of the web app was to help users find new ones, focusing on innate features, such as plot similarity, rather than simply using popularity.

1.4 Solution

Deep learning, a subset of machine learning, employs networks inspired by the human brain, efficiently processing complex and unstructured data. Unlike traditional machine learning, deep learning excels at hidden relationships and pattern discovery [4], enabling faster model training, deployment and implementation [5], making it a valuable asset for recommendation systems, such as AniBrain.

While the capabilities of AniBrain rely on user-provided initial data, LumiNovels seeks to overcome these limitations through deep learning algorithms and content-based filtering. By analysing detailed information from MyAnimeList and AniList APIs, LumiNovels provides finely-tuned, data-driven recommendations that address challenges such as the Cold Start problem, Grey Sheep users, limited content analysis, and sparsity. With this advanced approach, users can enjoy a more diverse and personalised set of recommendations, resulting in an improved reading experience.

2 Objectives

The objectives below ensure a robust recommendation system that enhances user experience and overcomes the limitations observed in AniBrain.

2.1 Primary Objectives

- Develop and integrate a **deep learning algorithm** capable of providing highly accurate light novel recommendations based on user preferences and content analysis.
- Leverage the **MyAnimeList** and **AniList APIs** to access light novels for algorithmic analysis and unique recommendations.
- Employ advanced **content-based-filtering** techniques to offer fine-tuned recommendations that align with their expectations and preferences.
- Create a robust **Java backend** that houses the powerful deep learning model and handles the processing of user queries and content analysis.
- Implement a user-friendly **React App** with **TypeScript** to serve as the **frontend** of LumiNovels.

2.2 Secondary Objectives

- **Enhance user engagement** through interactive elements, such as user reviews, ratings and the ability to discuss recommended light novels with an internal community of readers.
- Implement **real-time content updates** and recommendations for users.
- Ensure that LumiNovels **operates optimally**, delivering smooth interactions and swift response times to guarantee user satisfaction.
- **Refine the user interface** to an intuitive and visually appealing web app, facilitating ease of use, involving a responsive design and streamlined navigation.

2.3 Possible Extensions

- Implement **advanced natural processing** and **sentiment analysis** to consider user comments, reviews and light novel summaries when generating recommendations.
- Facilitate **social interactions** within the web app, allowing a community of users to share their favourite light novels, reviews and recommendations with friends and engage in discussions.
- Incorporate **advanced user profiling** techniques that adapt continuously to users' evolving preferences, ensuring tailored suggestions.

3 Technical Approach

3.1 Methodology

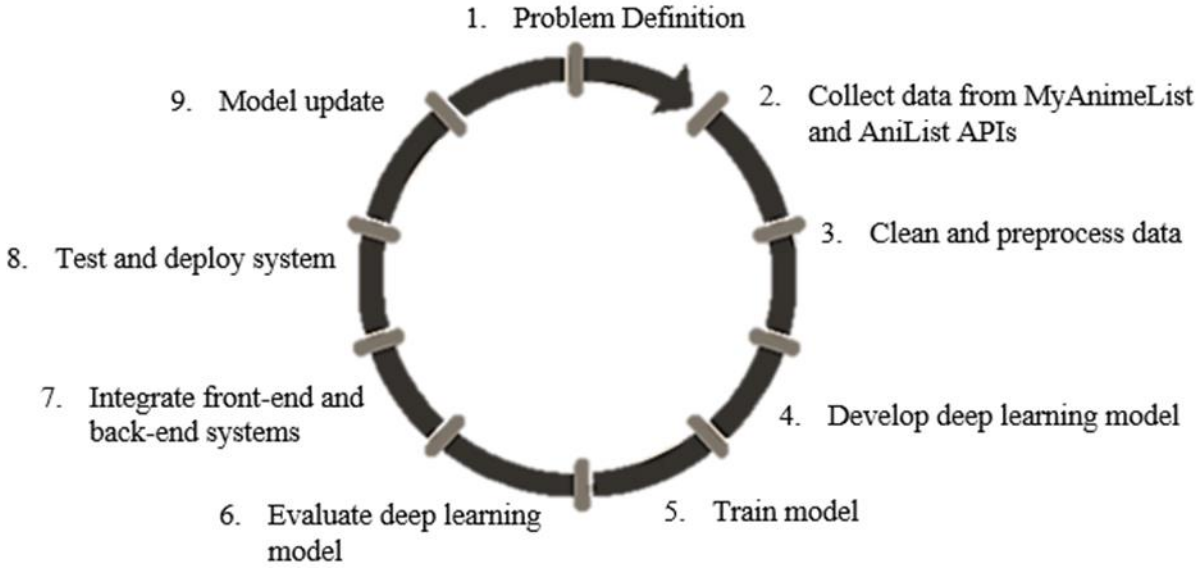


Figure 4. Development Methodology Diagram

LumiNovels will adhere to an agile software development methodology, prioritising collaboration and iterative progress. Such an approach is ideal for the project's intricate complexity and dynamic nature [6]. Regular feedback from stakeholders, especially users and project supervisors, will be integrated to guarantee that LumiNovels attains its goals.

The development of the model and integration will take place in weekly sprints, which will help manage its complexity and produce new versions of the code, uploading onto GitHub for streamlined version control. After each sprint, one day will go toward reviewing the research and completing courses which will aid in LumiNovels [7], for example, an AWS Machine Learning course on Approaching a Machine Learning Project.

3.2 Testing

Comprehensive testing will encompass various aspects, focusing on deep learning with a content-based filtering model and the LumiNovels web app. Testing of the deep learning algorithm will be initiated continuously by new components, data or model retraining. Unit tests ensure the correctness of individual model components, and regression tests detect and resolve previously encountered problems. Additionally, reproducibility tests assure consistent results during retraining or parameter adjustments. Moreover, multiple tests assess the deep learning model's robustness, including bias and noise checks. [8]

Integration tests will confirm that the composite parts of LumiNovels, the frontend and backend, work seamlessly to achieve the desired quality results. The iterative testing process will allow continuous improvement of both the deep learning model and the web app while maintaining quality and robustness throughout the development lifecycle.

4 Timeline

The Gantt chart below displays the estimated timeline for the LumiNovels project. Most sections have extended time frames due to possible complexities, and breaks exist to prevent burnout. The chart has several terminologies, including DOC (Documentation), DDLN (Deadline), BRK (Break), and PREP (Preparation). The diagram was created with the help of the onlinegantt website [9].

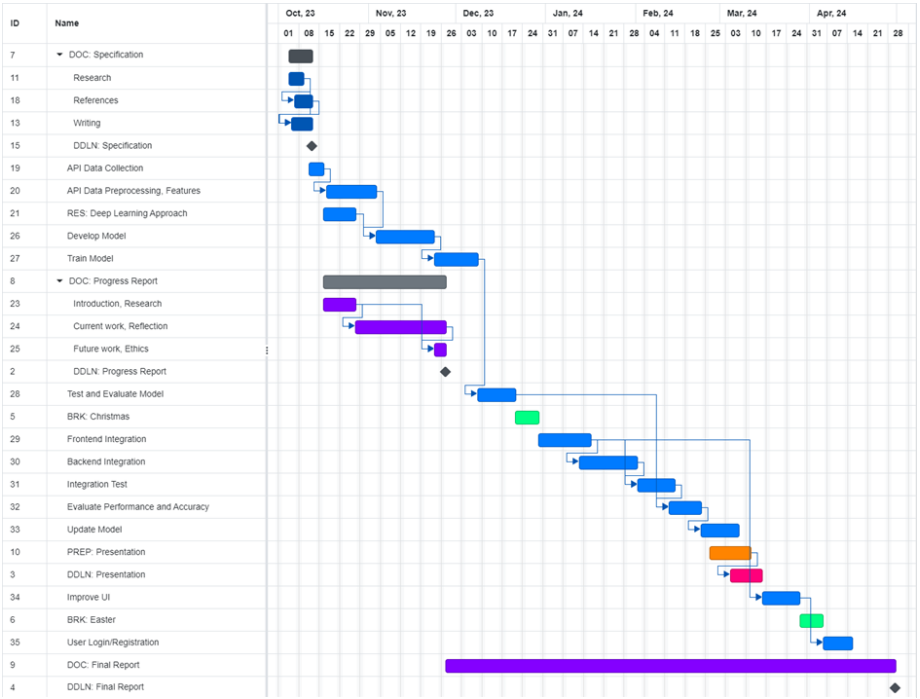


Figure 5. Gantt Chart for LumiNovels

5 Resource Requirements and Risks

5.1 Resources Involved

The deep learning model uses JDK 11, jupyter notebook on Python 3, and a Java kernel to

execute Java code on the jupyter notebook [10]. The data required to train the model is from open-source APIs such as MyAnimeList [11] and AniList [12]. Version control will be managed using GitHub, and the repository will remain private for security purposes.

The web app will use React with TypeScript for the frontend, and the backend will host the deep learning model in Java, as it has inherent performance advantages over Python. Java's excellent typing system ensures type safety and secure software development by preventing unintended object mutations; additionally, its early error detection during compilation contributes to more robust code [13]. While Python is the most commonly used language in machine learning, specifically deep learning, Java's speed, type safety, and error detection make it a compelling choice for projects like LumiNovels, as performance and security are critical.

5.2 Potential Risks

When building a deep learning model for light novel recommendations with content-based filtering using open-source APIs, there are several risks to consider:

- **Lack of Machine learning expertise:** inadequate knowledge in deep learning may lead to difficulties in model development.
- **Data quality and reliability:** dependency on APIs for data could lead to missing or incorrect information. An unreliable source can affect the accuracy of recommendations.
- **Software or Hardware failure:** the software may crash leading to the code being lost. Without any access to the code, it can lead to difficulties meeting the deadlines.

To mitigate these risks, it is essential to have backup plans in place.

- **Training and Skill development:** with the abundance of resources available online, short courses explaining deep learning concepts and approaches are available to overcome the skill gap.
- **Data quality control:** Implementing data preprocessing and cleaning pipelines to ensure the integrity and quality of data obtained from the APIs.
- **Version Control:** Saving the code in GitHub enables access to any computer, in the case the software or the laptop has any issues.

6 Ethical, Legal, Privacy and Security

The project does not involve any user data, therefore there are no legal concerns in that matter. In terms of APIs and Libraries used, they are all open-source and freely available for non-commercial use, therefore, there are no issues. The only data being accessed from the APIs are simply the titles, genres and average scores, possibly scoping out to using the synopsis of the book, but no personal data will be processed, so there are no ethical concerns.

7 References

1. Tyagi, N. (2019, October 26). 6 Dynamic Challenges in Formulating the Recommendation System. Analytics Steps. www.analyticssteps.com https://www.analyticssteps.com/blogs/6-dynamic-challenges-formulating-imperative-recommendation-system [Accessed 10-10-2023]

2. Khusro, S., Ali, Z., and Ullah, I. (2016). Recommender Systems: Issues, Challenges, and Research Opportunities. Information Science and Applications (ICISA) 2016, Volume 376, Pages 1182 - 1186. ISBN: 978-981-10-0556-5

3. About | AniBrain. (2022) AniBrain. www.anibrain.ai https://anibrain.ai/about [Accessed 9-10-2023]

4. What is Deep Learning? Deep Learning Explain – AWS. (n. d.). Amazon Web Services, Inc. www.aws.amazon.com https://aws.amazon.com/what-is/deep-learning [Accessed 11-10-2023]

5. Steck, H., Baltrunas, L., Elahi, E., Liang, D., Raimond, Y., and Basilico, J. (2021). Deep Learning for Recommender Systems: A Netflix Case Study. AI Magazine, 42(3), Pages 7 – 18. https://doi.org/10.1609/aimag.v42i3.18140 [Accessed 11-10-2023]

6. Cohn, M. (2011, January 15). Deciding What Kind of Projects are Most Suited for Agile. Mountain Goat Software. www.mountaingoatsoftware.com

   https://www.mountaingoatsoftware.com/blog/deciding-what-kind-of-projects-are-most-suited-for-agile [Accessed 12-10-2023]

7. González-Sanabria, J. S., Morente-Molinera, J. A., and Castro-Romero, A. (2017). DeSoftIn: A methodological proposal for individual software development. Revista Facultad de Ingeniería, 26(45), Pages 23-32. http://ww.scielo.org.co/scielo.php?pid=S0121-11292017000200023&script=sci_arttext [Accessed 11-10-2023]

8. Chorev, S. (2021, October 13). How to Test Machine Learning Models. Deepchecks. www.deepchecks.com https://deepchecks.com/how-to-test-machine-learning-models [Accessed 12-10-2023]

9. Free Online Gantt Chart Software. (n. d.) Onlinegantt. www.onlinegantt.com https://www.onlinegantt.com/#/gantt [Accessed 11-10-2023]

10. Installation – Dive into Deep Learning 0.1.0 documentation. (n. d.) D2l https://d2l.djl.ai/chapter_installation/index.html

11. MyAnimeList API (beta ver.). (n. d.). myanimelist.net https://myanimelist.net/apiconfig/references/api/v2#operation/manga_manga_id_get

12. Introduction - AniList APIv2 Docs. (2023). Gitbook.io https://anilist.gitbook.io/anilist-apiv2-docs/

13. Python vs. Java Performance (n. d.). SnapLogic. https://www.snaplogic.com/glossary/python-vs-java-performance#:~:text=Python%20and%20Java%20are%20two